Data Governance and Lineage

know what your data means, where it goes, and who depends on it

Watch the 4-min video

In one line: data governance makes data findable, trustworthy and safe: a **catalog** with owners and definitions, **lineage** showing how data flows from sources to tables, dashboards and models, **quality** checks shown where people look, and **tag-based access** with masking. Lineage turns "what breaks if I change this?" from a guess into a query.

1 Who uses this column?

The team plans to drop `customer_phone` from the main database. Someone asks which dashboards, reports and ML models use it. Nobody knows. It gets dropped, and the next morning a revenue dashboard is broken and a model's training job fails. Governance and lineage answer that question **before** the change, in minutes.

2 What governance is (and isn't)

Data governance is the set of practices that make data trustworthy and safe to use: who **owns** each dataset, what it **means**, how **good** it is, and who may **access** it. Without it, a growing company ends up with thousands of tables nobody understands, three definitions of "active customer", and numbers nobody trusts. Done well, it makes data **easier** to use: people find, understand and trust data faster, instead of asking around in chat. Done badly, it's a committee that slows everyone down; the modern approach is automation and clear ownership, not approval forms.

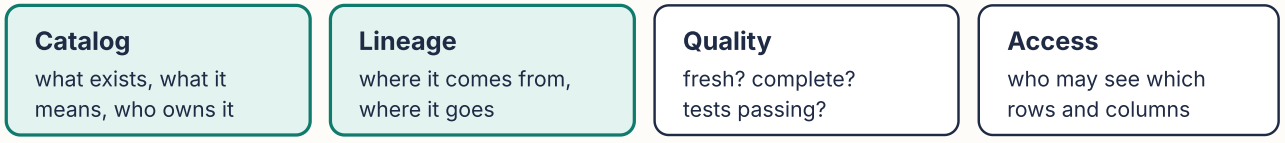


Figure 1. Four pillars. The catalog is where all four are shown to users.

Roles matter: a **data owner** (accountable, usually a business lead) approves access and definitions; a **data steward** maintains descriptions and quality rules; the platform team runs the tooling. A **data contract** between a producing team and its consumers (schema, meaning, freshness, change process) stops upstream changes from breaking downstream users silently.

3 The data catalog

A catalog (Microsoft Purview, Databricks Unity Catalog, DataHub, OpenMetadata, AWS Glue Data Catalog / DataZone, Google Dataplex, Collibra, Alation) lists every dataset with its schema, description, **owner**, tags (`PII`, `finance`, `certified`), freshness, quality status and usage statistics (who queries it, how often). Analysts search it like a search engine; popular, certified tables rise to the top, and deprecated ones are marked. Metadata is harvested automatically from databases, warehouses, pipelines and BI tools; humans add meaning (descriptions, glossary terms like "net revenue").

4 Lineage

Lineage records how data flows: this dashboard reads this gold table, which a dbt model builds from these silver tables, which come from this source column. **Table-level** lineage shows dataset dependencies; **column-level** lineage shows which output columns derive from which input columns, which is what you need for a column drop or PII tracking.

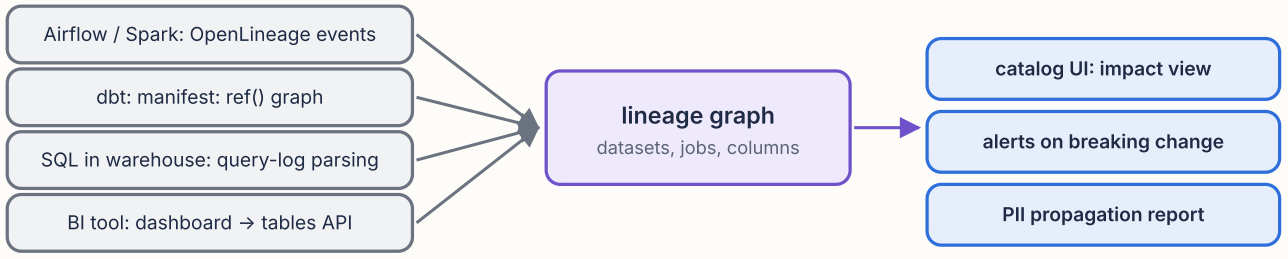


Figure 2. Lineage is captured automatically from where the work happens, then assembled into one graph.

- **OpenLineage** is an open standard: jobs emit run events naming their input and output datasets, with optional facets such as schema and column lineage. Integrations exist for Airflow, Spark, dbt and Flink; Marquez is the reference backend, and several catalogs accept the events.
- **dbt** knows its own graph: every `ref()` is an edge, published in its manifest.
- **SQL parsing** of warehouse query logs derives lineage, including columns, for SQL nobody instrumented.
- **BI connectors** map dashboards and reports to the tables and columns they query.

Gaps are common: Python notebooks, ad-hoc exports and spreadsheets. Treat unknown edges as a risk, not as "no dependencies".

5 Three uses of lineage

Use	Question	Direction in the graph
Impact analysis	What breaks if I change or delete this?	downstream
Root cause analysis	This number looks wrong: where did it come from, what changed?	upstream
Compliance	Where does this personal data end up, so we can protect, delete or report it?	downstream from PII sources

Lineage also propagates **tags**: a column derived from a PII column can inherit the PII tag automatically, so the access rules follow the data. That's how a deletion request or a breach assessment finds every copy (lesson 67).

6 Quality and access

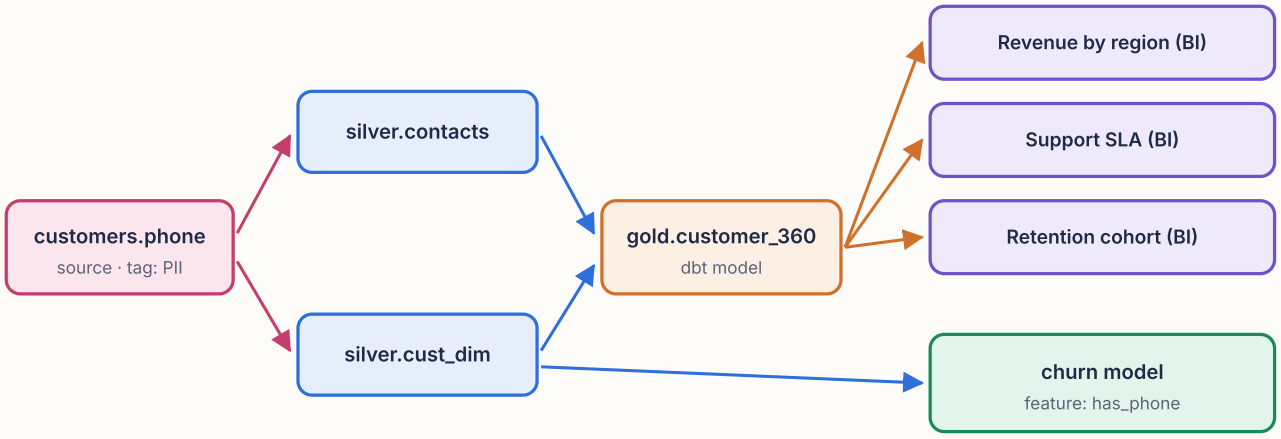
Quality checks run with each pipeline: freshness (updated in the last 2 hours?), volume (row count within the expected range?), schema (columns unchanged?), validity (`not_null`, `unique`, accepted values, referential integrity) and business rules. Tools: dbt tests, Great Expectations, Soda, and observability platforms. Results appear in the catalog, so users see "fresh, 42/42 tests passing" before they trust a table, and a failed check can stop a broken table from being published.

Access policies are best written against tags, not tables (attribute-based access control): "columns tagged PII are masked unless the user is in group `support-pii`"; "rows are filtered to the user's region".

Unity Catalog column masks and row filters, Snowflake tag-based masking policies, and Purview policies implement this. A new table with a PII column is protected the moment it's tagged, with no ticket.

WORKED EXAMPLE: TRACING A PII COLUMN BEFORE REMOVING IT

In the catalog, an engineer opens `customers.phone`, tagged PII. Column-level lineage shows it flows into **two silver tables**, **one gold table** (`customer_360`), **three dashboards** and **one ML feature** (`has_phone`) in the churn model. The catalog lists each asset's owner, and usage stats show one dashboard hasn't been viewed in 90 days. The owners are notified with a removal date. Two dashboards switch to a masked or derived column, the stale one is retired, and the data science team retrains the churn model without the feature (checking that accuracy holds). The column is marked **deprecated** for two weeks; the lineage view is re-checked to confirm no remaining consumers, and only then is it removed. Nothing breaks the next morning.



Impact of dropping customers.phone: 2 silver tables, 1 gold table, 3 dashboards, 1 ML feature, each with a named owner.

Figure 3. Column-level lineage for `customers.phone`: the impact list, with an owner for every node.

COMMON MISTAKES

- A catalog filled once by hand and never updated: metadata must be harvested automatically.
- Datasets without a named owner, so nobody answers questions or approves access.
- Table-level lineage only, when the change or the PII question is about one column.
- Access rules per table instead of per tag, so new copies of PII are unprotected.

7 Key terms

Data catalog: searchable inventory of datasets with owners and meaning

Column-level lineage: which output columns derive from which input columns

Data contract: agreed schema, meaning and change process between producer and consumers

Lineage: record of how data flows between datasets, jobs and reports

Data owner / steward: accountable approver / maintainer of a dataset

Tag-based access: policies that follow classification tags

8 Interview questions

Q1. Why does data lineage matter? BASIC

It shows how data flows from sources to tables, dashboards and models, which enables impact analysis before changes, root cause analysis when numbers look wrong, and compliance for personal data.

Q2. What else is part of data governance? BASIC

A catalog with owners and definitions, data quality checks, and tag-based access policies with masking.

Q3. How is lineage captured? BASIC

Automatically: OpenLineage events from orchestrators and Spark, dbt's dependency graph, parsing of warehouse SQL logs, and BI tool connectors.

Q4. Revenue on the dashboard dropped 20% overnight. How does governance help? SCENARIO

Walk lineage upstream from the dashboard, check freshness and test results at each step, and find the changed or failed source or model.

Q5. How do you make sure new copies of PII are protected? SCENARIO

Classify at the source, propagate tags through column-level lineage, and enforce access with tag-based masking policies rather than per-table grants.

Q6. Lineage shows no consumers for a column. Is it safe to drop? TRICKY

Not yet: notebooks, exports and external tools may be invisible to lineage; deprecate first, check query logs during a grace period, then drop.

Q7. Why do data contracts help when you already have lineage? TRICKY

Lineage tells you who will break; a contract prevents the break by making schema and meaning a versioned agreement with a change process.

Next: SD-V-019 · Edge Computing backendroadmap.com